

FocusDesk

Comprehensive Developer & Architecture Manual

An exhaustive, production-grade reference guide engineered for future maintainers, core engineers, and cross-platform architects. Details every layer, function call chain, state transition, procedural audio formula, and Android system integration.

ARCHITECTURAL STYLE Clean Architecture (Domain/Data/Presentation) + MVI	TARGET PLATFORMS Android SDK 26–35 & KMP iOS Framework Ready
LANGUAGE & COMPILER Kotlin 2.1.0 • Compose BOM 2025.01.00	LOCAL PERSISTENCE SQLite Room 2.6.1 + KSP Code Generation
AUDIO SYNTHESIS Real-Time 44.1 kHz PCM Procedural (Zero Assets)	STATUS BAR INTEGRATION MediaSession & Notification.MediaStyle Dynamic Island

1. Executive Overview & Philosophy

FocusDesk is a modern, production-grade productivity and deep-work companion engineered with **Kotlin Multiplatform (KMP)**, **Jetpack Compose**, and **Room Database**. It departs from conventional Pomodoro apps by eliminating static asset loops and third-party UI widgets in favor of native platform integration:

- **System-Native Dynamic Island:** Pins an active foreground `MediaSession` with `Notification.MediaStyle` to project live countdowns directly into the camera punch-hole island and status bar on modern Android devices (ColorOS/OxygenOS Aqua Dynamics, Xiaomi HyperOS, Samsung One UI, and Android 14/15 Live Notifications).
- **Procedural Ambient Audio Synthesizer:** Rather than bundling 60MB of pre-recorded audio files that suffer from audible loop seams, FocusDesk synthesizes stereo rain, Brownian noise, pink noise, coffee shop rumble, and forest birdsong mathematically at 44,100 Hz in real-time.
- **Offline-First Room Persistence:** All focus sessions, user settings, streak counters, and task priorities are saved locally in an embedded SQLite Room database with reactive Kotlin `Flow` queries and automatic schema seeding.
- **Tactile Spring Physics Design System:** Bespoke glassmorphism, 5 curated theme palettes, and spring curves calibrated 1:1 against Apple SwiftUI `.snappy`, `.smooth`, and `.bouncy` physics.

2. Why the Folder Structure is Laid Out Like This

The project is divided into two distinct Gradle modules: `:shared` and `:composeApp`. This structure strictly enforces the **Dependency Inversion Principle** and allows cross-platform portability.

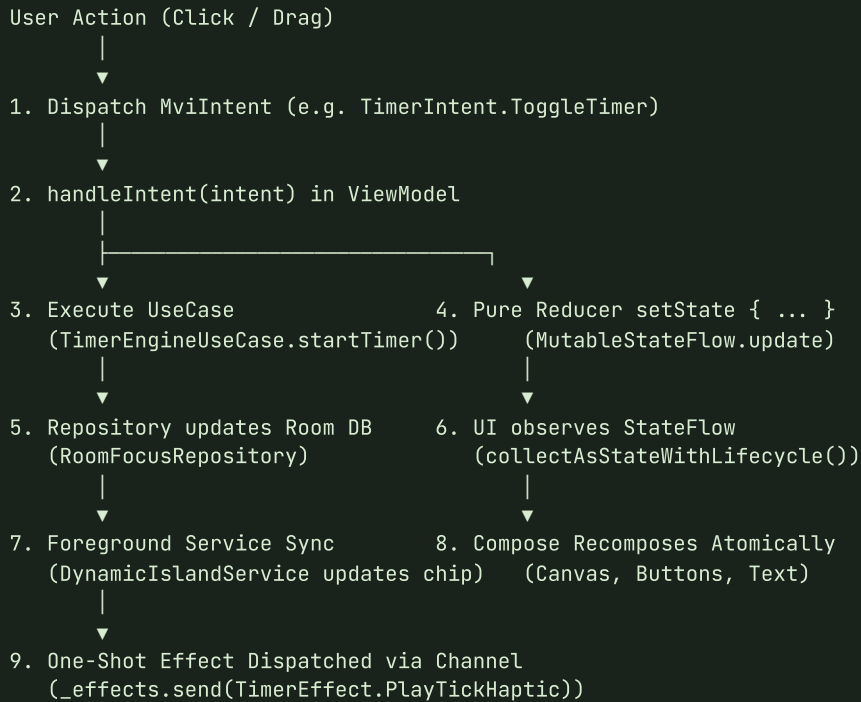
```
FocusDesk/
├── shared/                                     # Pure KMP Core Module (No Android UI dependencies)
│   └── src/
│       ├── commonMain/kotlin/com/focusdesk/    # Pure Kotlin (Runs on JVM, Android, and iOS)
│       │   ├── core/
│       │   │   ├── di/SharedModule.kt         # Platform-agnostic Koin bindings
│       │   │   ├── mvi/
│       │   │   │   └── platform/              # 'expect' declarations (AmbientAudioEngine, HapticEngine)
│       │   ├── data/repository/               # In-memory repository implementations (for unit tests & iOS)
│       │   │   └── domain/
│       │   │       ├── model/                 # Pure business entities (FocusSession, TaskItem, UserSetting)
│       │   │       ├── repository/           # Repository interfaces (FocusRepository, TaskRepository)
│       │   │       └── usecase/              # Use cases (TimerEngineUseCase, ManageTasksUseCase, GetAnaly
│       ├── androidMain/kotlin/com/focusdesk/   # Android 'actual' hardware bridges (AudioTrack, Vibrator)
│       └── iosMain/kotlin/com/focusdesk/       # iOS 'actual' hardware bridges (AVAudioEngine, CoreHaptics)
└── composeApp/                                # Android Application Module
    └── src/main/
        ├── AndroidManifest.xml                # Foreground service, permissions, & broadcast receivers
        ├── kotlin/com/focusdesk/app/
        │   ├── FocusDeskApplication.kt        # Application subclass configuring Koin dependency graph
        │   ├── MainActivity.kt                # Root activity & navigation backstack coordinator
        │   ├── data/
        │   │   ├── local/
        │   │   │   ├── FocusDeskDatabase.kt  # Room Database class with singleton & prepopulation callback
        │   │   │   ├── dao/
        │   │   │   │   ├── FocusSessionDao, TaskDao, UserSettingsDao
        │   │   │   └── entity/
        │   │   │       ├── Room entities & toDomain() / toEntity() mappers
        │   │   └── repository/               # Room implementations of domain repositories
        │   ├── di/AppModule.kt              # Android Koin module (Room DB, DAOs, ViewModels, AudioEngine
        │   ├── presentation/                # Jetpack Compose UI Layer (grouped by feature)
        │   │   ├── analytics/               # Insights screen, charts, session history
        │   │   ├── designsystem/           # Themes, Color palettes, Typography, Shapes, Motion, Sliders
        │   │   ├── focus/                  # Quick-start cards & new session bottom sheet
        │   │   ├── home/                   # Dashboard, streak counter, daily progress ring
        │   │   ├── navigation/             # Bottom navigation bar & Screen routes
        │   │   ├── onboarding/            # First-run guided interactive setup
        │   │   ├── settings/              # Timer interval sliders, sounds, theme picker
        │   │   ├── sounds/                 # Ambient soundscapes & audio mixer
        │   │   └── timer/                  # Pomodoro timer, circular progress, complete screen
        │   └── service/                    # Android System Services
        │       ├── DynamicIslandService.kt   # Foreground service with MediaSession & Notification.MediaSt
        │       └── DynamicIslandActionReceiver.kt # Broadcast receiver for notification remote actions
```

2.1 Key Design Questions Answered

Design Decision	Why It Is Structured This Way
Why is <code>FocusRepository</code> in <code>shared</code> while <code>RoomFocusRepository</code> is in <code>composeApp</code> ?	Adheres to Clean Architecture's Dependency Inversion . The Domain layer defines the business contract (<code>FocusRepository</code>). The concrete implementation depends on Android SQLite Room APIs, which belong in the Android-specific <code>composeApp</code> module.
Why use <code>expect / actual</code> for Audio and Haptics?	Audio output (<code>AudioTrack</code> on Android vs <code>AVAudioEngine</code> on iOS) and Haptics (<code>VibratorManager</code> on Android vs <code>UIFeedbackGenerator</code> on iOS) require low-level platform APIs. Declaring <code>expect class AmbientAudioEngine</code> allows the shared Domain layer to trigger audio without caring what OS is underneath.
Why separate <code>presentation/</code> subfolders by feature instead of by layer?	Feature-First Packaging ensures high cohesion. All components relating to the timer (<code>TimerScreen</code> , <code>TimerViewModel</code> , <code>TimerContract</code> , <code>ActiveFocusSessionCard</code>) live together in <code>presentation/timer/</code> , making navigation and code review immediate.

3. Architecture Paradigm: Clean Architecture + MVI

FocusDesk implements a strict **Model-View-Intent (MVI)** pattern powered by Kotlin Coroutines and `StateFlow`. All ViewModels inherit from the base `MviViewModel<I, S, E>`:



3.1 The MVI Contract Classes

- `MviState` : Marker interface. Each screen has exactly one immutable data class representing 100% of its visual state (e.g. `TimerState` , `SettingsState` , `AnalyticsState`).
- `MviIntent` : Marker interface representing all possible user or system intentions (e.g. `TimerIntent.ToggleTimer` , `TimerIntent.AddFiveMinutes` , `TimerIntent.ConfirmReset`).
- `MviEffect` : Marker interface for one-off transient events consumed once by the UI (e.g. `ShowSnackbar` , `PlayCelebrationHaptic` , `PlaySoundEffect`). Backed by a buffered coroutine `Channel` to guarantee zero dropped events during screen rotations.

4. Domain Layer Deep Dive

4.1 TimerEngineUseCase — Function Usages & Call Chains

Located in `shared/src/commonMain/kotlin/com/focusdesk/domain/usecase/TimerEngineUseCase.kt` . This is the central brain of FocusDesk.

Function Signature	Why It Is Called & Call Chain	Parameters, Returns & Concurrency Guards
<code>observeSession(): Flow<FocusSession></code>	Called by <code>TimerViewModel</code> and <code>DynamicIslandService</code> in their initialization scopes. Subscribes to real-time session updates.	Returns: Reactive <code>Flow<FocusSession></code> from the repository. Threading: Emits on background or main thread safely.
<code>startTimer()</code>	Called when user taps the Play button on <code>TimerScreen</code> , <code>ActiveFocusSessionCard</code> , or the lock screen notification action.	Transitions status from <code>Idle</code> or <code>Paused</code> to <code>Running</code> . Sets <code>startedAtTimestamp</code> on first run. Resets <code>lastTickEpochMs</code> .
<code>pauseTimer()</code>	Called when user taps Pause button or receives phone call / lockscreen pause event.	Transitions status to <code>Paused</code> . Freezes countdown without modifying <code>remainingSeconds</code> .
<code>tick(force: Boolean = false): FocusSession</code>	CRITICAL CORE FUNCTION. Invoked once per second by the ViewModel's ticking coroutine loop. Decrements <code>remainingSeconds</code> by 1.	850ms Debounce Guard: Uses <code>lastTickEpochMs</code> to enforce a minimum 850ms interval between ticks. This strictly prevents double-ticking if UI and Foreground Service tick simultaneously. If timer reaches 0, transitions status to <code>Completed</code>, records session via <code>focusRepository.recordCompletedSession()</code>, and stops audio.
<code>resetTimer()</code>	Called after user confirms reset in <code>ExitConfirmationDialog</code> .	Fetches configured duration from <code>SettingsRepository</code> and resets remaining/target seconds. Sets status back to <code>Idle</code> .
<code>switchMode(mode: SessionMode)</code>	Called when user taps Work, Short Break, or Long Break mode tabs.	Loads corresponding duration from <code>UserSettings.intervals</code> and sets status to <code>Idle</code> .
<code>skipSession()</code>	Called when user clicks Skip button.	Calls private <code>determineNextMode()</code> : If currently in <code>Work</code> and <code>currentRound ≥ totalRounds</code> , transitions to <code>LongBreak</code> ; otherwise <code>ShortBreak</code> .
<code>startCustomSession(...)</code>	Called from the <code>NewSessionBottomSheet</code> or quick-start presets on <code>HomeScreen</code> .	Creates new <code>FocusSession</code> with custom title, category, mood, duration in minutes, and soundscape. Starts immediately in <code>Running</code> status.

<code>addSeconds(seconds: Long)</code>	Called when user taps the "+5 min" button during an active focus session.	Adds <code>300L</code> to both <code>remainingSeconds</code> and <code>targetDurationSeconds</code> , preserving the progress calculation accurately.
<code>endSessionEarly(): FocusSession</code>	Called when user confirms "End Session" in dialog or notification action.	Marks status as <code>Completed</code> , records session in Room, stops soundscape, and returns the completed session object.
<code>toggleAudioMute(...): Soundscape</code>	Called when user taps Mute/Unmute in UI or notification action.	If playing, saves active soundscape to settings and sets session to <code>None</code> . If muted, restores saved soundscape. Preserves user preference.

5. Data Layer & Offline-First Persistence Deep Dive

5.1 Room Database (FocusDeskDatabase.kt)

FocusDesk embeds an offline-first SQLite database built with AndroidX Room 2.6.1:

- **Entities:** `FocusSessionEntity`, `UserSettingsEntity`, `TaskEntity`.
- **Database Prepopulation:** Utilizes `RoomDatabase.Callback.onCreate()` to insert initial settings (ID = 1) and four starter tasks on first launch, ensuring the app is never in an uninitialized empty state.

5.2 Streak & XP Gamification Algorithm (RoomFocusRepository.kt)

When a focus session is recorded as completed, `RoomFocusRepository` executes an atomic transaction:

```
val nowMs = System.currentTimeMillis()
val currentEpochDay = nowMs / (1000L * 60 * 60 * 24)

val newStreak = when {
    settings.lastActiveEpochDay == currentEpochDay → settings.dayStreak.coerceAtLeast(1)
    settings.lastActiveEpochDay == currentEpochDay - 1 → settings.dayStreak + 1
    else → 1 // Gap detected: streak broken, reset to day 1
}
val bestStreak = maxOf(settings.bestStreak, newStreak)
val sessionMinutes = ((session.targetDurationSeconds - session.remainingSeconds).coerceAtLeast(60L) / 60)
val pointsEarned = maxOf(sessionMinutes, 6) // Base XP + duration bonus
val newXp = settings.xp + pointsEarned
val newLevel = (newXp / 100) + 1

userSettingsDao.updateXpAndStreak(newXp, newLevel, newStreak, bestStreak, currentEpochDay)
```

5.3 Weekly 7-Day Bucket Distribution Algorithm

`observeProductivityStats()` uses Kotlin Coroutines `combine()` across sessions, tasks, and settings to build the 7-day Monday-to-Sunday bar chart in real-time:

- Calculates Monday 00:00:00 timestamp using `Calendar.getInstance().apply { firstDayOfWeek = Calendar.MONDAY }`.
- Allocates 7 buckets: `[0 = Monday, 1 = Tuesday, 2 = Wednesday, 3 = Thursday, 4 = Friday, 5 = Saturday, 6 = Sunday]`.
- Iterates through sessions completed this week and accumulates `focusMinutes` and `sessionsCount` into the matching bucket.

6. Hardware, Sound Synthesis & System Services Deep Dive

6.1 Procedural Ambient Audio Synthesizer (AmbientAudioEngine)

Located in `shared/src/androidMain/kotlin/com/focusdesk/core/platform/AndroidAudioPlayer.kt`.

Why Procedural Synthesis?

Pre-recorded audio files consume 60+ MB of storage and suffer from audible loop seams. FocusDesk calculates audio samples mathematically in real-time at 44.1 kHz 16-bit PCM, producing seamless infinite soundscapes with a

0 MB bundle size

.

Soundscape	Synthesis Mathematical Model & Implementation Details
Rain & Heavy Rain	1st-order IIR low-pass filtered white noise (<code>filterState = filterState * 0.94 + white * intensity</code>). A Poisson-distributed random drop generator inserts micro-sine bursts with randomized pitch (1,000–3,000 Hz) with exponential amplitude decay.
White, Pink, Brown Noise	Gaussian random walk generator (<code>random.nextGaussian() * 0.35</code>). Filter coefficients dictate the frequency slope: <code>BrownNoise = 0.96</code> (-6 dB/octave warm deep rumble), <code>PinkNoise = 0.92</code> (-3 dB/octave equal energy per octave), <code>WhiteNoise = 0.86</code> (crisp uniform energy).
Coffee Shop & Library	Dual low-frequency sinusoidal fundamentals at 65 Hz and 130 Hz simulate HVAC ventilation and background refrigerator hums, blended with stochastic filtered chatter noise.
Forest & Birdsong	0.25 Hz sine envelope modulates background wind turbulence. A periodic 3-second cycle triggers multi-tone bird chirps peaking at 2,400 Hz with frequency modulation.
Ocean	0.10 Hz sinusoidal swell envelope modulates filtered surf noise, simulating rhythmic crashing ocean waves on a shoreline.

6.2 Dynamic Island & Foreground Service (DynamicIslandService)

Located in `composeApp/src/main/kotlin/com/focusdesk/app/service/DynamicIslandService.kt`.

- **Native Android Dynamic Island Integration:** Modern Android firmware (OnePlus/Oppo ColorOS Aqua Dynamics, Xiaomi HyperOS, Samsung One UI Now Bar, Android 14/15) projects active `MediaSession` tokens with `Notification.MediaStyle` directly around the front camera punch-hole island.
- **Dynamic Canvas Badge Rendering:** `createTimerBadgeBitmap()` renders an off-screen 256×256 pixel circular progress ring with remaining MM:SS time in bold monospaced typography, battery-efficient dark background, and glowing progress arcs.
- **Lockscreen Remote Controls:** Dispatches broadcast intents to `DynamicIslandActionReceiver` to pause/resume, toggle mute, or terminate sessions directly from the lock screen.

7. Presentation & UI Layer Deep Dive

7.1 Single-Activity Coordinator (MainActivity.kt)

`MainActivity` coordinates the entire screen graph with state-driven visibility:

- **Full-Screen Timer:** Rendered when `isSessionActive && !isTimerScreenMinimized` with vertical slide transition.
- **Minimized Docked Card:** Rendered when `isSessionActive && isTimerScreenMinimized`, docking `ActiveFocusSessionCard` above the bottom bar with playback and mute controls.
- **Idle Inactivity Compression:** If the screen is untouched for 8 seconds, `isBottomBarIdleCompressed` collapses the navigation bar. Any touch gesture instantly restores it.

7.2 The 5 Bespoke Theme Palettes (Color.kt)

Palette Name	Primary Color	Accent / Glow	Dark Card BG	Aesthetic Mood
Default Focus	Forest Olive (#4A683F)	Lime Glow (#D4F384)	Deep Forest (#16251C)	Clean, modern organic focus
Paper Studio	Terracotta (#8E532B)	Amber (#E5A169)	Warm Espresso (#291D16)	Tactile warm craft paper
Low Tide	Sea Teal (#2B5C56)	Sage Mint (#9EE0D4)	Abyssal Teal (#142927)	Calm, aquatic serenity
Last Light	Rust Red (#B55739)	Peach Coral (#F5AC95)	Dark Coral (#2C1D1A)	Sunset warmth, deep immersion
Night Bloom	Deep Violet (#5D4786)	Soft Lilac (#C0A6EB)	Midnight Plum (#211A2C)	Nocturnal deep work focus

8. Dependency Injection Graph (Koin)

FocusDesk uses **Koin 4.0.2** for clean, lightweight dependency injection across multiplatform and Android modules:

```
sharedModule:
├─ domainModule
│   ├── single { TimerEngineUseCase(get(), get(), get()) }
│   ├── single { ManageTasksUseCase(get()) }
│   └── single { GetAnalyticsUseCase(get()) }
└─ inMemoryDataModule (for Unit Tests & iOS)
    ├── single<FocusRepository> { InMemoryFocusRepository() }
    ├── single<TaskRepository> { InMemoryTaskRepository() }
    └── single<SettingsRepository> { InMemorySettingsRepository() }

appModule (Android composeApp):
├─ single { FocusDeskDatabase.getInstance(androidContext()) }
├─ single { get<FocusDeskDatabase>().focusSessionDao() }
├─ single { get<FocusDeskDatabase>().userSettingsDao() }
├─ single { get<FocusDeskDatabase>().taskDao() }
├─ single<FocusRepository> { RoomFocusRepository(get(), get(), get()) }
├─ single<SettingsRepository> { RoomSettingsRepository(get()) }
├─ single<TaskRepository> { RoomTaskRepository(get()) }
├─ single { AmbientAudioEngine(get()) }
├─ viewModel { TimerViewModel(get(), get(), get()) }
├─ viewModel { AnalyticsViewModel(get()) }
└─ viewModel { SettingsViewModel(get(), get()) }
```

9. Next Developer Handbook: How-To Recipes & Gotchas

9.1 How to Add a New Soundscape in 3 Steps

1. Open `UserSettings.kt` and add the enum entry: `Campfire("Campfire", isPremium = false, category = "Nature")`.
2. Open `AndroidAudioPlayer.kt` and implement the synthesis formula inside the `when (currentSoundscape)` loop.
3. **Finished!** The soundscape immediately appears on the Sounds screen, bottom sheet, and quick-start cards.

9.2 Common Gotchas & Solutions

Gotcha 1: Concurrency & Fast Ticking

Never bypass the 850ms guard in `TimerEngineUseCase.tick()`. Multiple coroutines (ViewModel and Foreground Service) observe the session; without this guard, the timer could run at double speed.

Gotcha 2: AudioTrack Main Thread Blocking

Never write audio samples to `AudioTrack` on the Android UI Main thread. Audio generation must always occur on the dedicated background daemon thread (`FocusDesk-AudioSynth`).

Gotcha 3: Jetpack Compose State Lifecycles

Always use `collectAsStateWithLifecycle()` instead of `collectAsState()` to ensure Room database flows pause when the application is minimized, preserving battery life.